

# C++ Experiment Processing Code

```
#include <iostream>
#include <string>
#include <fstream>
#include <vector>
#include <iomanip>
#include <algorithm>

using namespace std;

struct experiment {
    string treatment;
    double result;
};

// --- Function Prototypes ---
double gMx(vector<experiment> exp);
double gMn(vector<experiment> exp);
vector<double> gMMN(vector<experiment> exp);
vector<double> diffExpRes(vector<experiment> exp1, vector<experiment> exp2);
vector<experiment> readExperiment(string expName);
double median(vector<experiment> list);

int main() {
    // Part 1: Initial Data Loading
    vector<experiment> exp1;
    exp1 = readExperiment("Experiment1.txt");

    // Part 2: Normalization (gMMN)
    vector<double> res1 = gMMN(exp1);

    cout << "Normalized Results (exp1): " << endl;
    for (int i = 0; i < res1.size(); i++) {
        cout << setprecision(3) << res1.at(i) << (i == res1.size() - 1 ? "" : ", ");
    }
    // Expected Output: 0.44, 0.58, 1.00, 0.35, 0.00, 0.63, 0.26, 0.88, 0.81, 0.92

    // Part 3: Set Difference
    vector<experiment> exp2 = readExperiment("Experiment2.txt");

    // TODO: Call the set difference function (2pts)
    vector<double> resDiff = _____;

    cout << "\n\nSet Difference (exp1 vs exp2): " << endl;
    for (int i = 0; i < resDiff.size(); i++) {
        cout << setprecision(4) << resDiff.at(i) << (i == resDiff.size() - 1 ? "" : ", ");
    }
    // Expected Output: 12.65, 16.36, 13.14, 9.83, 14.72, 15.63

    return 0;
}

// --- Implementation Section ---
// 1. Find Maximum
```

```

double gMx(vector<experiment> exp) {
    double expRes = exp.at(0).result;
    for (int i = 1; i < exp.size(); i++) {
        if (exp.at(i).result > expRes) expRes = exp.at(i).result;
    }
    return expRes;
}

// 2. Find Minimum
double gMn(vector<experiment> exp) {
    double expRes = exp.at(0).result;
    for (int i = 1; i < exp.size(); i++) {
        if (exp.at(i).result < expRes) expRes = exp.at(i).result;
    }
    return expRes;
}

// 3. Min-Max Normalization
vector<double> gMMN(vector<experiment> exp) {
    double Mx = gMx(exp);
    double Mn = gMn(exp);
    vector<double> x;
    for (int i = 0; i < exp.size(); i++) {
        x.push_back((exp.at(i).result - Mn) / (Mx - Mn));
    }
    return x;
}

// 4. Set Difference Function (Handwritten on Page 1)
vector<double> diffExpRes(vector<experiment> exp1, vector<experiment> exp2) {
    vector<double> xRes;
    int flagCtr;
    int i, j;
    for (i = 0; i < _____; i++) {
        _____; // TODO: Initialize flag

        for (j = 0; j < _____; j++) {
            // TODO: Comparison logic (4pts)
            if (_____) {
                _____;
            }
        }
        if (_____) {
            // TODO: Push back result (4pts)
            _____;
        }
    }
    return xRes;
}

// 5. Median Function (Handwritten on Page 2)
double median(vector<experiment> list) {
    /* TODO: Logic to sort and find median */
    _____; // double minimum
    _____; // vector sorted
    while (!list.empty()) {

```

```

        _____; // find min
        _____; // push to sorted
    for(int i = 0; i < list.size(); i++) {
        if (_____ ) {
            _____; // remove from original
            break;
        }
    }
}

if (_____ ) { // check if even
    return (_____ ) / 2;
} else {
    return _____; // return middle
}
}

```

// 6. File Reading Function (Handwritten on Page 1)

```

vector<experiment> readExperiment(string expName) {
    ifstream experimentRec;

    _____; // TODO: Open file

    experiment tempExp;
    vector<experiment> vecExp;

    if (experimentRec.is_open()) {
        while (!experimentRec.eof()) {
            // TODO: Getline for treatment name (4pts)
            _____;

            experimentRec >> tempExp.result;

            // TODO: Ignore newline/buffer (4pts)
            _____;

            vecExp.push_back(tempExp);
        }
    } else {
        cout << "Failed to open file: " << expName << endl;
    }
    experimentRec.close();
    return vecExp;
}

```